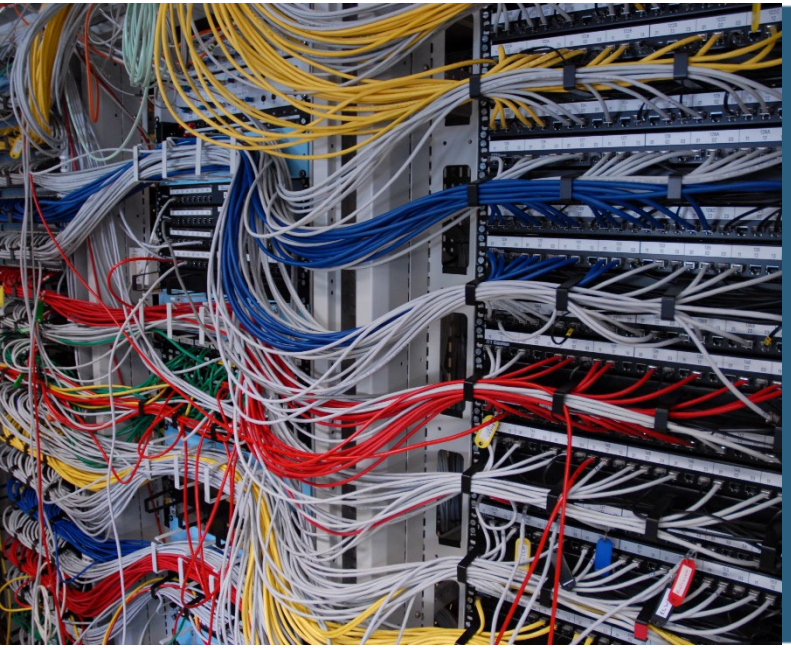
Terraform a OpenTofu

Čo nás čaká

- Terraform
- OpenTofu
- OpenStack provider
- Úlohy



Terraform

Čo je Terraform?

- Bývalá open-source Infrastructure as a code (IaaC)
  - Stojí za ním spoločnosť HashiCorp
- CLI nástroj používajúci skripty – manifesty
  - Prípona .tf
- Používa jazyk HCL (HashiCorp Configuration Language)
- Vie komunikovať s viacerými hypervízormi/prostrediami cez „providers“
  - Google Cloud
  - Amazon AWS
  - Microsoft Azure
  - VMware ESXi
  - OpenStack
  - Kubernetes
  - ...

Inštalácia

- Windows
  - Terraform je binárny súbor, ktorý je potrebné stiahnuť a spustiť
  - Nie je potrebná inštalácia
    - Ideálne pridať do PATH (napr. C:\Windows)
- Linux (Debian, RedHat)
  - Pridať repozitár a nainštalovať ako štandardný balíček
  - Alebo rovnako ako Win – jeden binárny súbor
- macOS, FreeBSD, OpenBSD, Solaris

- https://www.terraform.io/downloads

4 základné CLI príkazy

- terraform init
  - Inicializuje pracovný priečinok (working directory)
  - Prvý príkaz po vytvorení manifestu
- terraform plan
  - Ukáže, čo by chcel urobiť
  - Preview
- terraform apply
  - Vykoná veci definované v manifestoch
- terraform destroy
  - Vymaže vytvorené/modifikované veci
- terraform fmt
  - Naformátuje .tf súbory (učeše ich)

terraform init

```
# Prázdny priečinok

C:\Users\marek.KIS\terraform>terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files.
You may begin working with Terraform immediately by creating
Terraform configuration files.

C:\Users\marek.KIS\terraform>
```

Príklad – nič, len pripojenie

```
terraform {
required_version = ">= 0.14.0"
  required_providers {
    openstack = {
      source  = "terraform-provider-openstack/openstack"
      version = "~> 1.35.0"
    }
  }
}

provider "openstack" {
  user_name   = "user1"
  tenant_name = "project1"  # Názov projektu
  password    = "user1"
  auth_url    = "http://158.193.153.30:5000/v2.0"
  region      = "RegionOne"
}
```

terraform init

```
C:\Users\marek.KIS\terraform>terraform init
Initializing the backend...

Initializing provider plugins...
- Finding terraform-provider-openstack/openstack versions matching
"~> 1.35.0"...
- Installing terraform-provider-openstack/openstack v1.35.0...
- Installed terraform-provider-openstack/openstack v1.35.0 (self-
signed, key ID 4F80527A391BEFD2)

...

Terraform has been successfully initialized!
...

C:\Users\marek.KIS\terraform>
```

terraform plan

```
C:\Users\marek.KIS\terraform>terraform plan

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your
configuration and found no differences, so no
changes are needed.

C:\Users\marek.KIS\terraform>
```

Príklad – vytvorenie VM

```
variable "image" {
  type    = string
  default = "CirrOS"
}

variable "flavor" {
  type    = string
  default = "m1.tiny"
}

variable "network" {
  type    = string
  default = "internal"
}

resource "openstack_compute_instance_v2" "vm-terraform" {
  name        = "vm-terraform"
  image_name  = var.image
  flavor_name = var.flavor
  network {
    name = var.network
  }
}
```

terraform plan

```
C:\Users\marek.KIS\terraform>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

  # openstack_compute_instance_v2.vm-terraform will be created
  + resource "openstack_compute_instance_v2" "vm-terraform" {
      + access_ip_v4        = (known after apply)
      + access_ip_v6        = (known after apply)
      + all_metadata        = (known after apply)
      + all_tags            = (known after apply)
      + availability_zone   = (known after apply)
      + flavor_id           = (known after apply)
      + flavor_name         = "m1.tiny"
      + force_delete        = false
      + id                  = (known after apply)
      + image_id            = (known after apply)
      + image_name          = "CirrOS"
      + name                = "vm-terraform"
      + power_state         = "active"
      + region              = (known after apply)
      + security_groups     = (known after apply)
      + stop_before_destroy = false

      + network {
          + access_network = false
          + fixed_ip_v4    = (known after apply)
          + fixed_ip_v6    = (known after apply)
          + floating_ip    = (known after apply)
          + mac            = (known after apply)
          + name           = "ext-net"
          + port           = (known after apply)
          + uuid           = (known after apply)
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these
actions if you run "terraform apply" now.

C:\Users\marek.KIS\terraform>
```

terraform apply

```
...
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

terraform apply

```
C:\Users\marek.KIS\terraform>terraform apply
...

  Enter a value: yes

openstack_compute_instance_v2.vm-terraform: Creating...
openstack_compute_instance_v2.vm-terraform: Still creating... [10s elapsed]
openstack_compute_instance_v2.vm-terraform: Creation complete after 13s
[id=b54ec017-fbc4-47e4-bc5c-4227d9b15b63]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

C:\Users\marek.KIS\terraform>
```

terraform destroy

```
C:\Users\marek.KIS\terraform>terraform destroy
...
  Enter a value: yes

openstack_compute_instance_v2.vm-terraform: Destroying... [id=b54ec017-
fbc4-47e4-bc5c-4227d9b15b63]
openstack_compute_instance_v2.vm-terraform: Still destroying...
[id=b54ec017-fbc4-47e4-bc5c-4227d9b15b63, 10s elapsed]
openstack_compute_instance_v2.vm-terraform: Destruction complete after 11s

Destroy complete! Resources: 1 destroyed.

C:\Users\marek.KIS\terraform>
```

OpenTofu

Čo je OpenTofu?

- Open-source Infrastructure as a code (IaaC)
  - Fork Terraform-u
- CLI nástroj používajúci skripty – manifesty
  - Prípona .tf
- Používa jazyk HCL (HashiCorp Configuration Language)
- Vie komunikovať s viacerými hypervízormi/prostrediami cez „providers“
  - Google Cloud
  - Amazon AWS
  - Microsoft Azure
  - VMware ESXi
  - OpenStack
  - Kubernetes
  - ...

Inštalácia

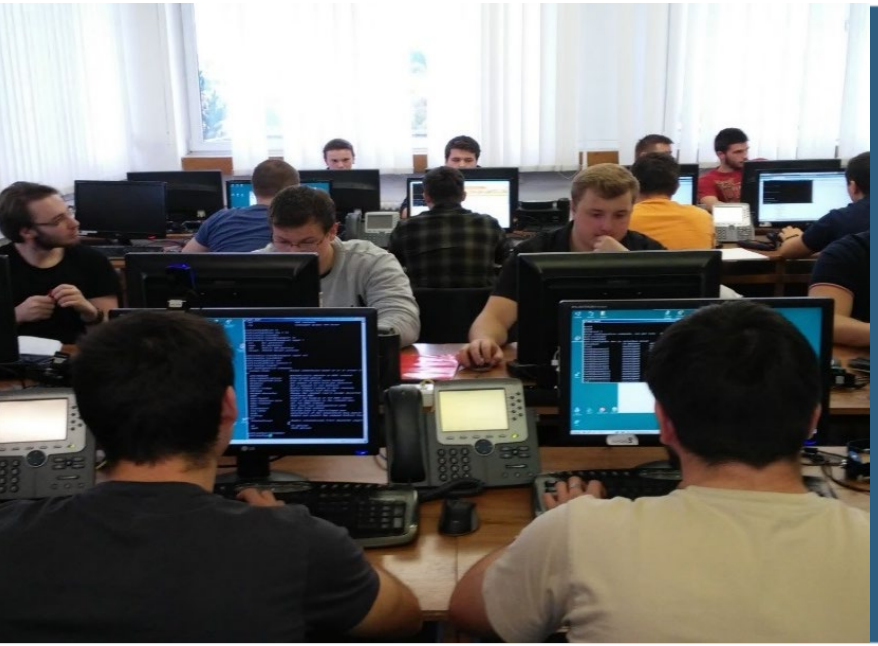
- Windows
  - CMD alebo PowerSwhell (nie sú potrebné admin práva)
  - winget install --exact --id=OpenTofu.Tofu

- Linux (Debian, RedHat), macOS, FreeBSD, OpenBSD, Solaris, Windows
  - Použiť inštalátor

- https://opentofu.org/docs/intro/install/

4 základné CLI príkazy

- tofu init
  - Inicializuje pracovný priečinok (working directory)
  - Prvý príkaz po vytvorení manifestu
- tofu plan
  - Ukáže, čo by chcel urobiť
  - Preview
- tofu apply
  - Vykoná veci definované v manifestoch
- tofu destroy
  - Vymaže vytvorené/modifikované veci
- tofu fmt
  - Naformátuje .tf súbory (učeše ich)



OpenStack provider

Terraform vs OpenStack HOT

```
resource "openstack_compute_instance_v2" "server" {
  name            = "meno_servera"
  image_name      = "Cirros"
  flavor_name     = "m1.nano"
}
```

```
resources:
  server:
    type: OS::Nova::Server
    properties:
      name: meno_servera
      image: Cirros
      flavor: m1.nano
```

Dokumentácia

- https://registry.terraform.io/providers/terraform-provider-openstack/openstack/latest/docs

Premenné = variables

- Môžu byť v ktoromkoľvek .tf súbore
- Pokiaľ nie je definovaná „default“ hodnota, používateľ je pri spustení skriptu (terraform apply) vyzvaný na zadanie hodnoty
- Typy
  - string
  - number
  - list(number)
  - list(string)
  - ...

```
variable "image" {
  type    = string
  default = "CirrOS"
}
```

openstack_compute_instance_v2

```
resource "openstack_compute_instance_v2" "basic" {
  name            = "basic"
  image_id        = "ad091b52-742f-469e-8f3c-fd81cadf0743"
# image_name      = "Cirros"
  flavor_id       = "3"
# flavor_name     = "m1.nano"
  key_pair        = "my_key_pair_name"
  security_groups = ["default"]
  network {
    name = "my_network"
  }
}
```

openstack_compute_instance_v2 – user data

```
resource "openstack_compute_instance_v2" "basic" {
  user_data        = "#cloud-config\nhostname:
basic.example.com\nfqdn: basic.example.com"
}


resource "openstack_compute_instance_v2" "basic" {
  user_data        = file("scripts/first-boot.sh")
}
```

openstack_compute_keypair_v2

```
resource "openstack_compute_keypair_v2" "test-keypair1" {
  name = "my-keypair1"
}



resource "openstack_compute_keypair_v2" "test-keypair2" {
  name       = "my-keypair2"
  public_key = "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQDAjpC1hwiOCCmKEWxJ4qzTTsJbKzndLotBCz5Pc
wtUnflmU+gHJtWMZKpuEGVi29h0A/+ydKek1O18k10Ff+4tyFjiHDQAnOfgWf7+b1yK+q
Dip3X1C0UPMbwHlTfSGWLGZqd9LvEFx9k3h/M+VtMvwR1lJ9LUyTAImnNjWG7TaIPmui3
0HvM2UiFEmqkr4ijq45MyX2+fLIePLRIF61p4whjHAQYufqyno3BS48icQb4p6iVEZPo4
AE2o9oIyQvj2mx4dk5Y8CgSETOZTYDOR3rU2fZTRDRgPJDH9FWvQjF5tA0p3d9CoWWd2s
6GKKbfoUIi8R/Db1BSPJwkqB"
}
```

openstack_networking_network_v2

```
resource "openstack_networking_network_v2" "network_1" {
  name           = "network_1"
  admin_state_up = "true"
}
```

openstack_networking_subnet_v2

```
resource "openstack_networking_subnet_v2" "subnet_1" {
  name       = "tf_test_subnet"
  network_id = "${openstack_networking_network_v2.network_1.id}"
  cidr       = "192.168.199.0/24"
  ip_version = "4" # alebo 6
}
```

openstack_networking_secgroup_v2

```
resource "openstack_networking_secgroup_v2" "secgroup_1" {
  name        = "secgroup_1"
  description = "My neutron security group"
}
```

openstack_networking_secgroup_rule_v2

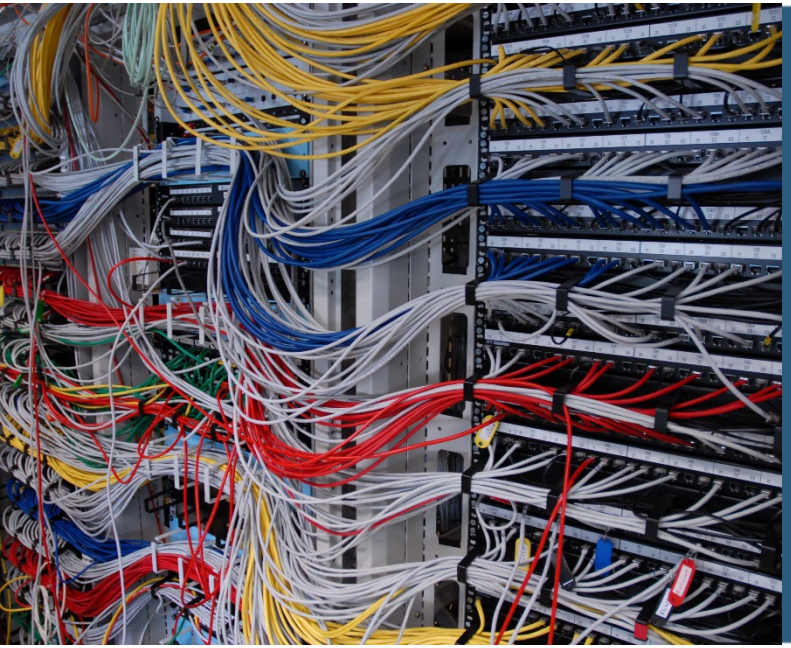
```
resource "openstack_networking_secgroup_rule_v2"
"secgroup_rule_1" {
  direction         = "ingress"
  ethertype         = "IPv4"
  protocol          = "tcp"
  port_range_min    = 22
  port_range_max    = 22
  remote_ip_prefix  = "0.0.0.0/0"
  security_group_id =
"${openstack_networking_secgroup_v2.secgroup_1.id}"
}
```

openstack_networking_router_v2

```
resource "openstack_networking_router_v2" "router_1" {
  name                = "my_router"
  admin_state_up      = true
  external_network_id = "f67f0d72-0ddf-11e4-9d95-e1f29f417e2f"
}
```

openstack_networking_router_interface_v2

```
resource "openstack_networking_router_interface_v2"
"router_interface_1" {
  router_id = "${openstack_networking_router_v2.router_1.id}"
  subnet_id = "f67f0d72-0ddf-11e4-9d95-e1f29f417e2f"
}
```



Úlohy pre vás

OpenStack KIS 2024

```
provider "openstack" {
  user_name   = "iiks0"
  tenant_name = "iiks0"
  password    = "integracia"
  insecure    = "true"
  auth_url    = "https://158.193.152.44:5000/v3/"
  region      = "RegionOne"
  user_domain_name = "admin_domain"

  endpoint_overrides = {
    "network"  = "https://158.193.152.44:9696/v2.0/"
    "compute"  = "https://158.193.152.44:8774/v2.1/"
    "identity" = "https://158.193.152.44:5000/v3/"
    "image"    = "https://158.193.152.44:9292/"
  }
}
```

Úloha 1

- Vytvorte skript, v ktorom budú 2 VM v spoločnej sieti.

- Parametre:
  - Názov siete: ext-net-154
  - Availability zone: nova
  - Flavor: 1c05r1d
  - Image: cirros
  - Security group: podľa uváženia

Úloha 2

- Vytvorte skript, v ktorom budú 2 VM. Jedna bude v externej sieti, druhá za ňou pripojená.

- Parametre:
  - Názov externej siete : ext-net-154
  - Názov internej siete : net-iiks#
  - Názov internej podsiete : subnet-iiks#
  - Availability zone: nova
  - Flavor: 1c05r1d
  - Image: cirros
  - Security zone: podľa uváženia

Úloha 3

- Vytvorte skript, v ktorom budú 2 VM. Jedna bude v externej sieti, druhá za ňou pripojená. Na prvom stroji povoľte smerovanie, na druhom nainštalujte web server. Vytvorte security group len pre služby na VM a SSH len z UNIZA siete (158.193.0.0/16).

- Parametre:
  - Názov externej siete : ext-net-154
  - Názov internej siete : net-iiks#
  - Názov internej podsiete : subnet-iiks#
  - Availability zone: nova
  - Flavor: 1c05r1d
  - Image: cirros
  - Security zone: podľa uváženia

Ďakujem za pozornosť